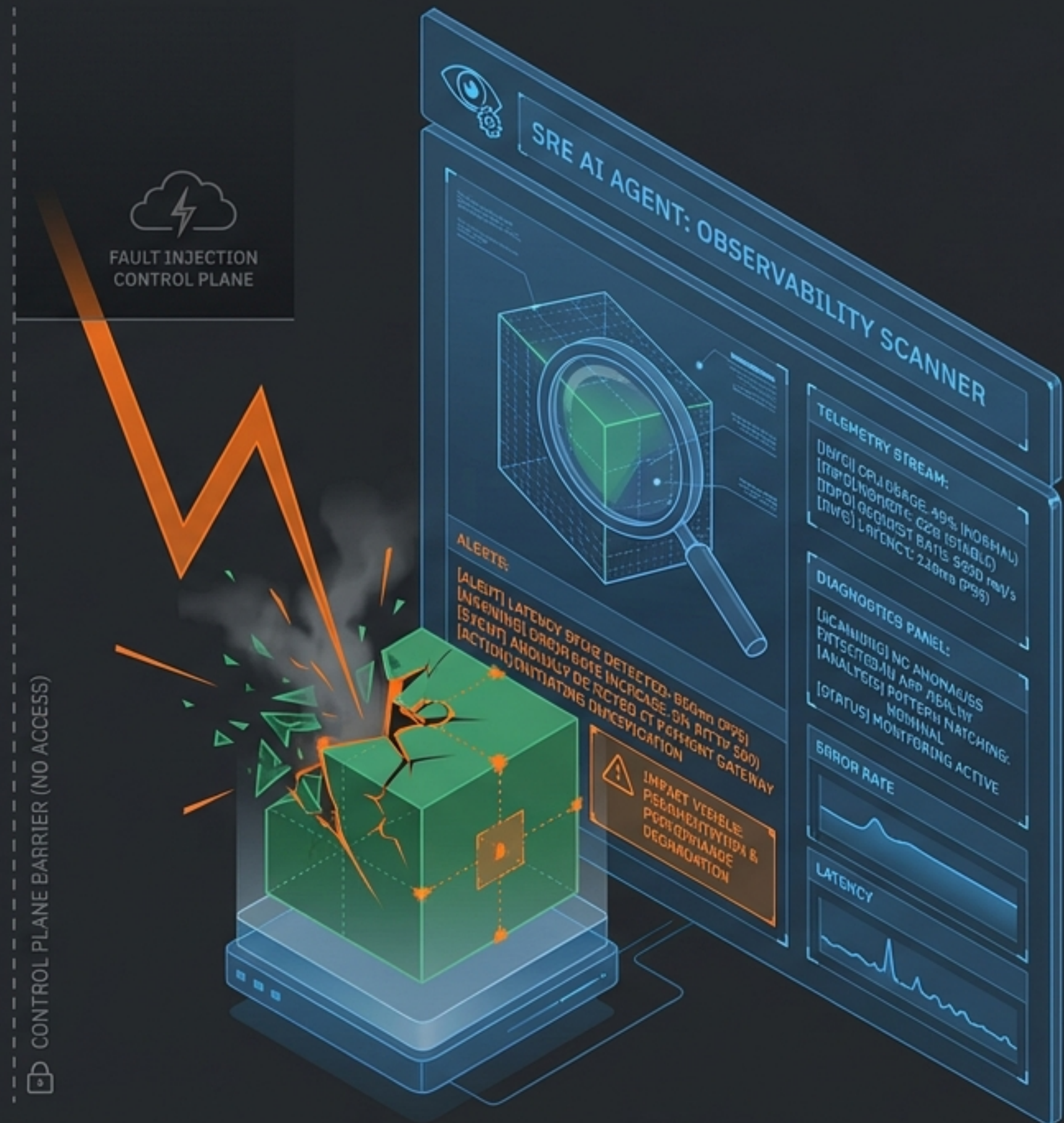
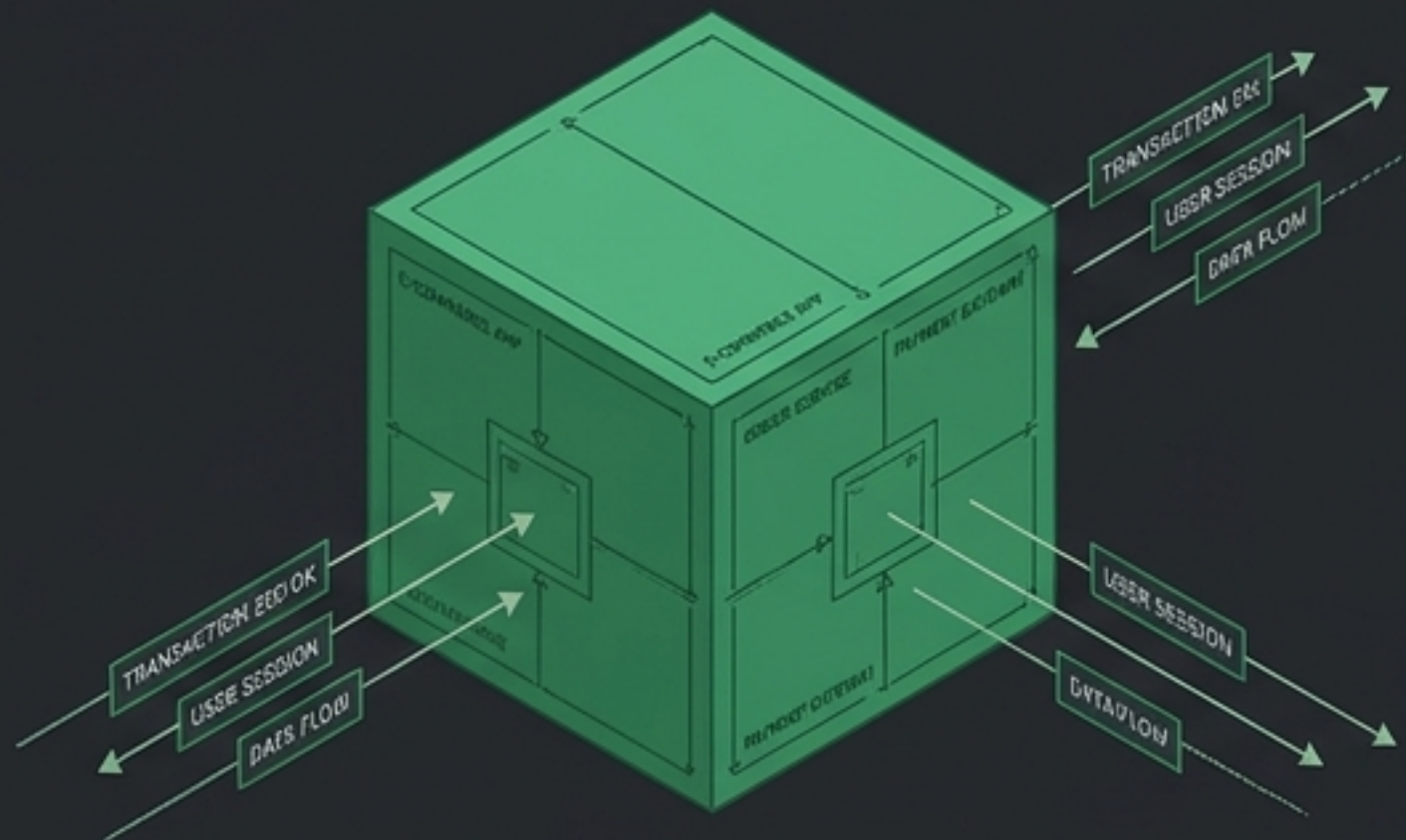


Лаборатория контролируемого хаоса для обучения SRE-агентов.

Идеальная мишень для машинного интеллекта

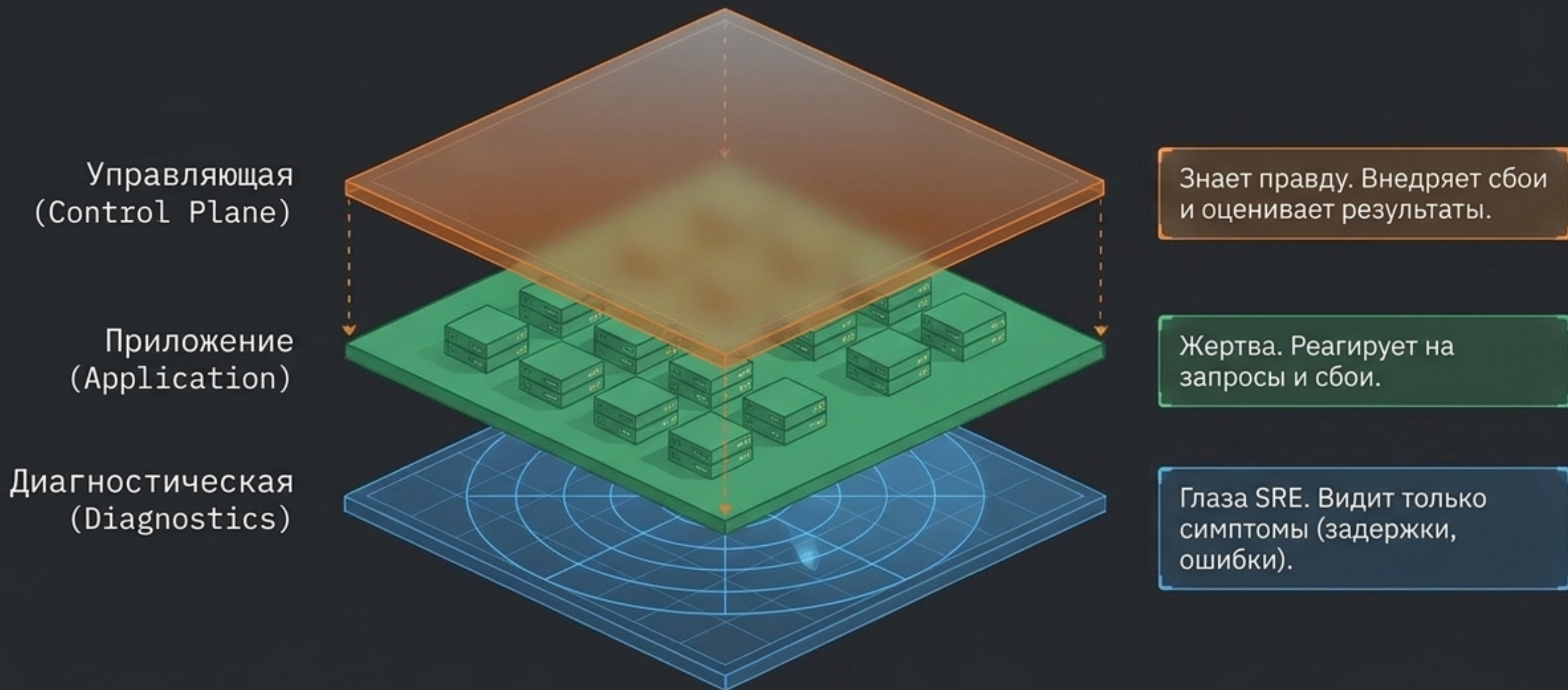
Agentic Testbed — это распределенная e-commerce система, которую мы намеренно и контролируемо ломаем.

Цель: научить SRE-агента расследовать инциденты исключительно по внешним симптомам и телеметрии, без доступа к управляющей плоскости.



Три плоскости реальности: Анатомия слепой зоны

Главное правило стенда: Управляющая плоскость абсолютно невидима для диагностической. Логи инъектора не попадают в систему мониторинга.



Быстрый запуск и развертывание полигона

17 контейнеров. 3 минуты до полной боевой готовности.

Требования: 4 CPU, 6GB RAM, Docker Compose v2, Make, Python3.

make up – сборка и запуск системы.
make smoke – проверка статуса.

[✓] ok app: ready

[✓] ok prometheus targets up

Радары и Порты

Внешний доступ:

Приложение
8080

Изолированная сеть (127.0.0.1):

Grafana
:3000

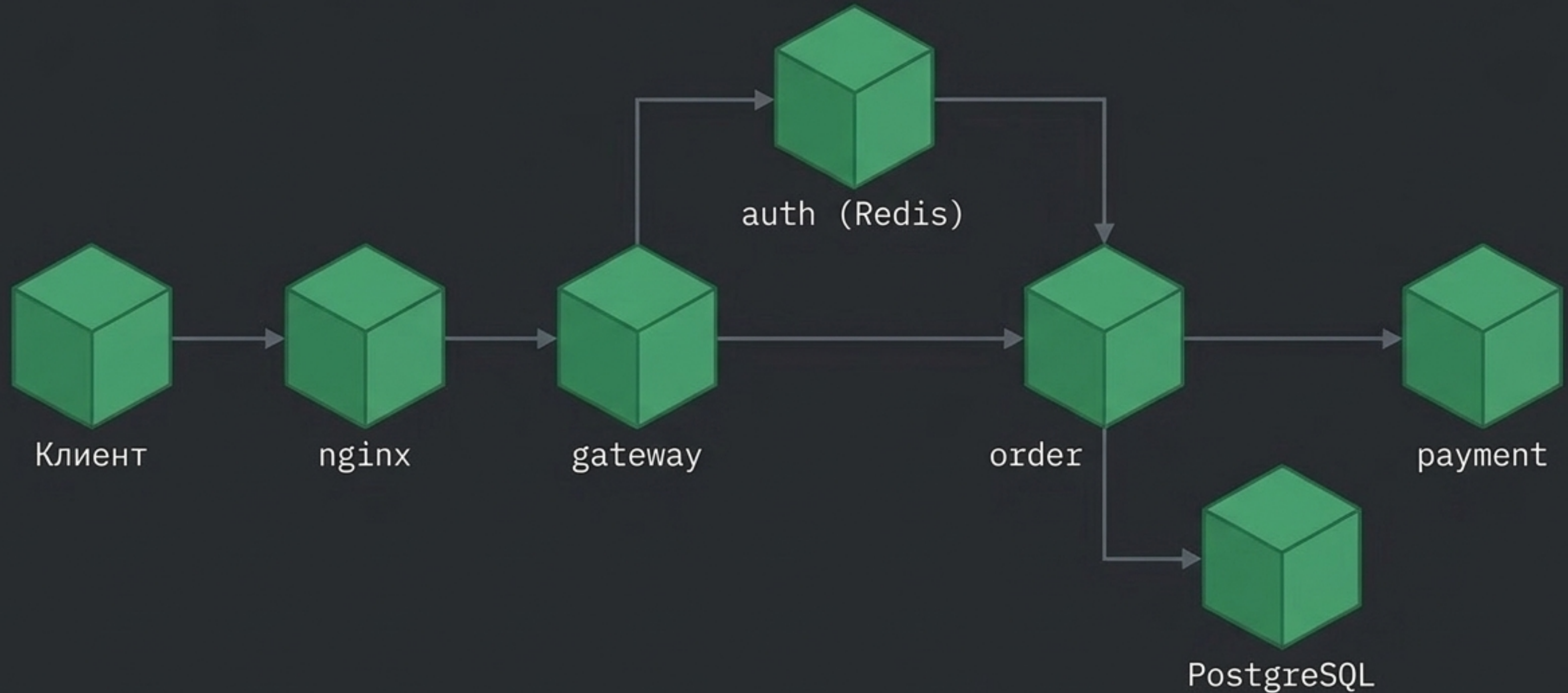
Prometheus
:9090

Fault Injector
:8090

Runner
:8091

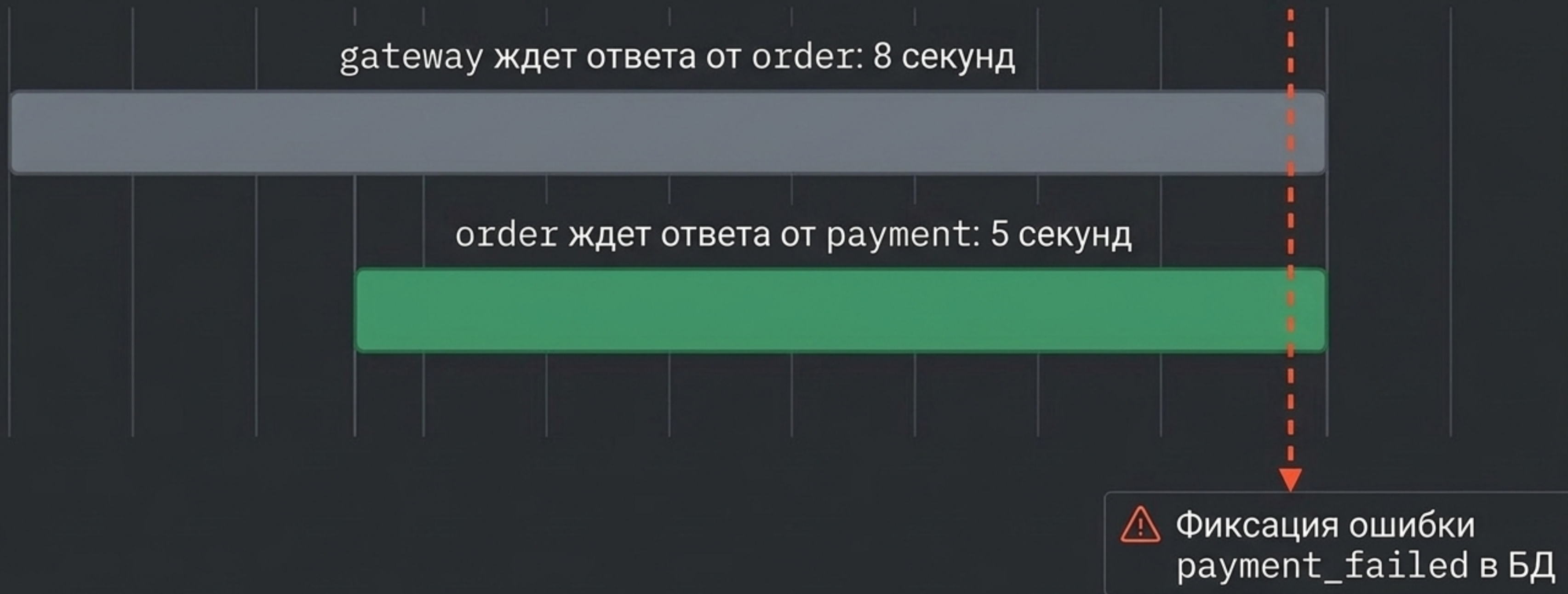
Топология жертвы: Маршрут клиентского запроса

Классическая микросервисная архитектура с вложенными зависимостями.



Матрешка таймаутов: Кто сдастся первым?

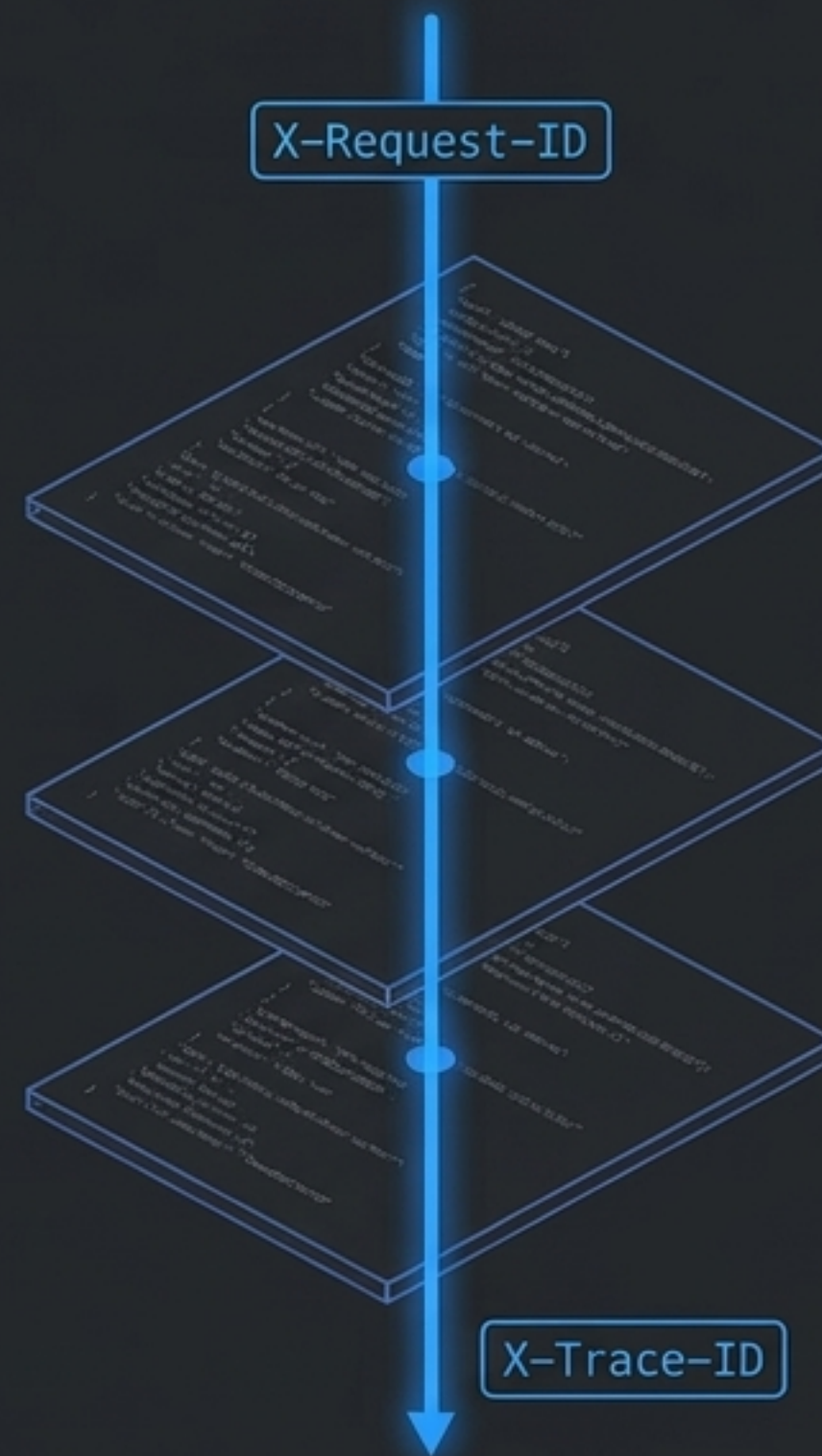
Вложенные таймауты гарантируют, что внутренний сервис успеет записать исход заказа до того, как внешний шлюз оборвет соединение.



Анатомия взаимодействия: API, Коды и Нити Ариадны

POST /orders

```
{  
  "customer_id": "user_abc123",  
  "items": [  
    { "sku": "ITEM-890", "qty": 2 }  
  ],  
  "payment_token": "tok_xyz789"  
}
```



Панель управления SRE: Радары Grafana

5 главных дашбордов для локализации проблемы

1. Service Overview

2. Dependencies

3. Logs

4. Traces

5. Diagnostic Domains

1. Service Overview:

RED-метрики, ресурсы.

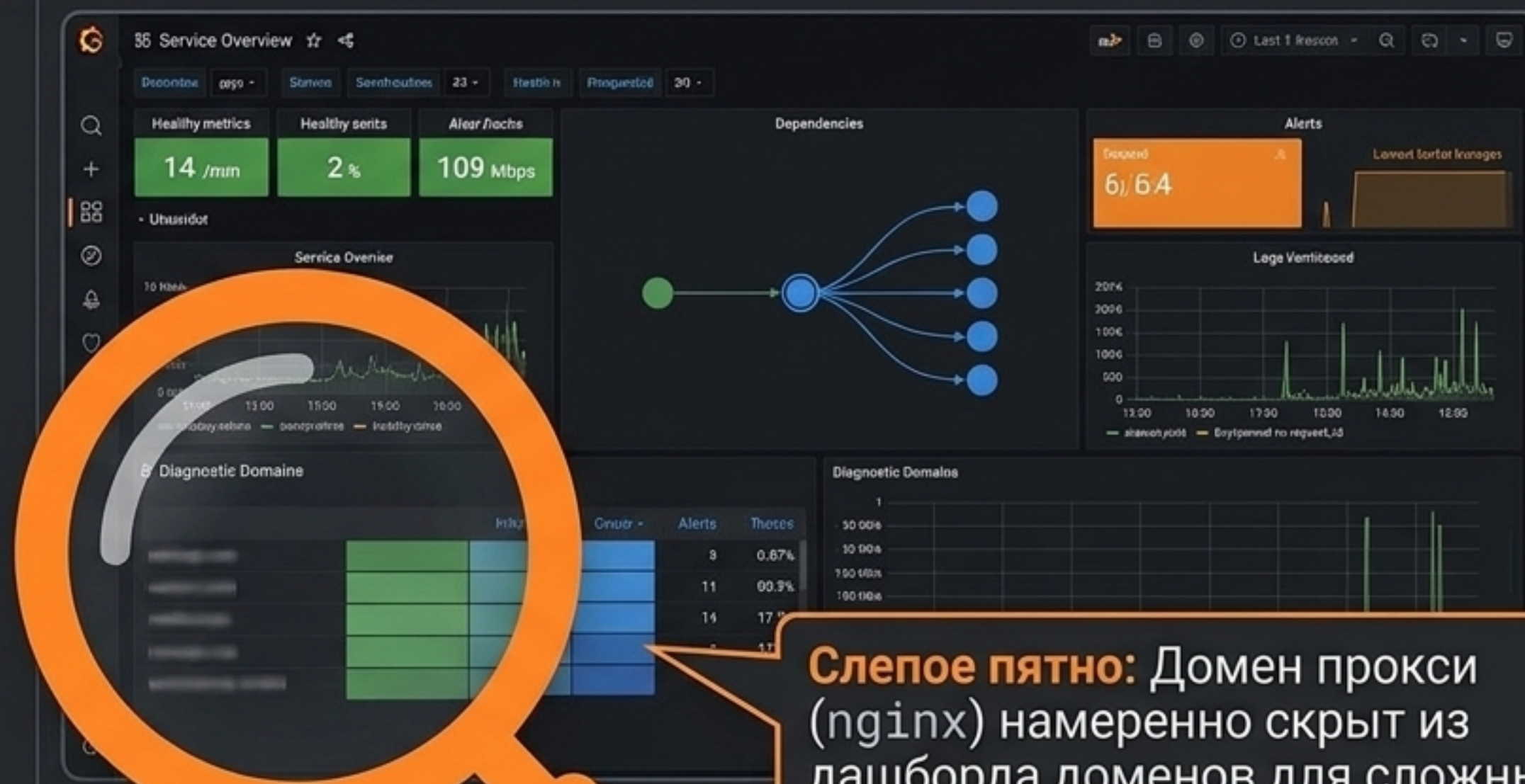
2. Dependencies: Латентность и ошибки ребер вызовов.

3. Logs: Агрегация сообщений по request_id.

4. Traces: Графы вызовов и узкие места.

5. Diagnostic Domains:

Тепловая карта по слоям.



Слепое пятно: Домен прокси (nginx) намеренно скрыт из дашборда доменов для сложных model-breaking сценариев.

Синтез сигналов: Loki, Prometheus, Tempo и Нагрузка

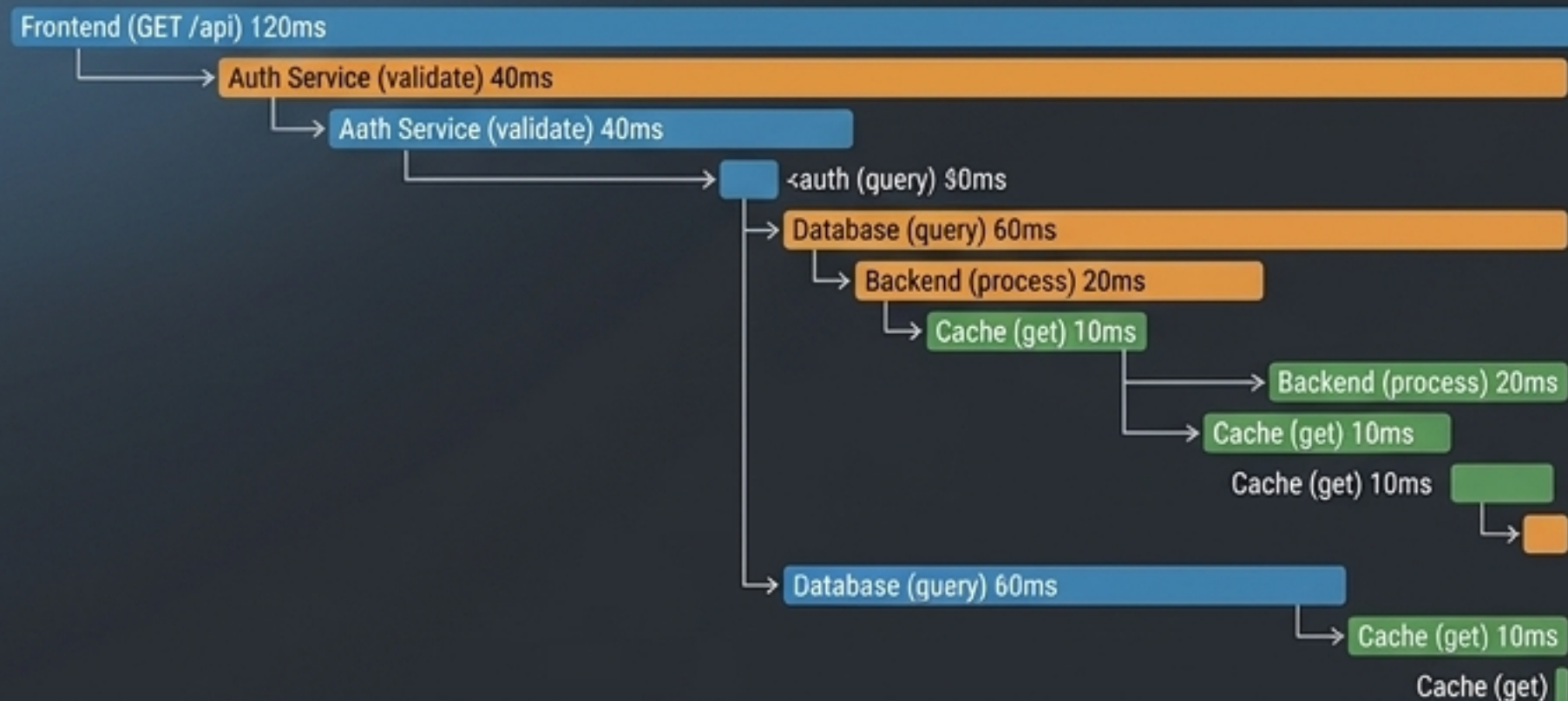
PromQL: Поиск доли 5xx или p95 гистограммы



LogQL: Поиск ошибок или сквозной поиск по request_id

```
{\"level\": \"error\", \"request_id\": \"abc...\", \"msg\": \"...\"}  
{\"level\": \"error\", \"request_id\": \"abc...\", \"msg\": \"...\"}  
{\"level\": \"error\", \"request_id\": \"abc...\", \"msg\": \"...\"}  
{\"level\": \"error\", \"request_id\": \"abc...\", \"msg\": \"...\"}
```

Темпо: Граф вызова со спанами



make load RATE=10 DURATION=300

Пuls системы: Графики мертвы без трафика

Механика разрушения: Управляющая плоскость

Бескомпромиссное удержание:

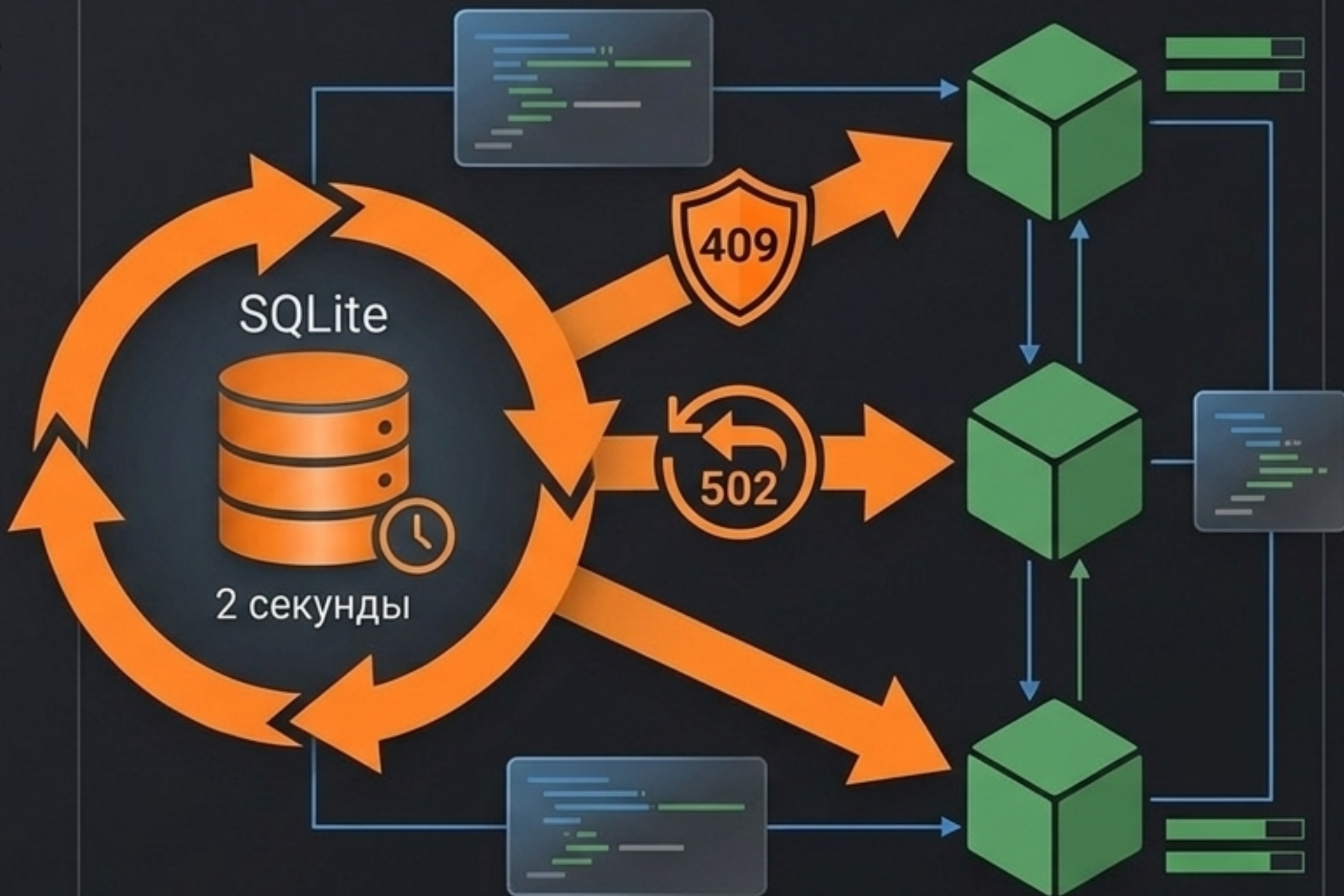
Фолты записываются в SQLite и переприменяются каждые 2 секунды.

Контроль конфликтов (409):

Защита от наложения фолтов на одну цель.

Частичные откаты (502):

При сбое внедрения система автоматически откатывает изменения.



Периодическая система фолтов: 22 вектора атаки

Приложение



payment-latency
payment-error
intermittent-errors
service-unavailable
dependency-timeout
redis-unavailable

Ресурсы



cpu-saturation
cpu-limit
memory-pressure

Базы Данных



db-slow-query
db-connection-exhaustion
db-lock-contention
redis-latency

Сеть



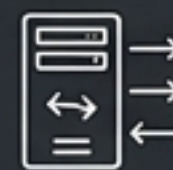
network-latency
packet-loss
connection-failure

Деплой и Конфиг



incorrect-endpoint
incorrect-timeout
bad-configuration
bad-deployment

Прокси



proxy-rate-limit
proxy-bandwidth-limit

Управление фолтами: Терминал оператора хаоса

Инъекция через Make-обертку

```
make fault TYPE=payment-latency  
PARAMS='{"latency_ms":2500}'
```

```
make fault TYPE=network-latency  
TARGET=order PARAMS='{"delay_ms":300}'
```

Прямое управление через API

```
POST /faults/validate
```

```
POST /faults
```

```
DELETE /faults/{id}
```

Очистка полигона

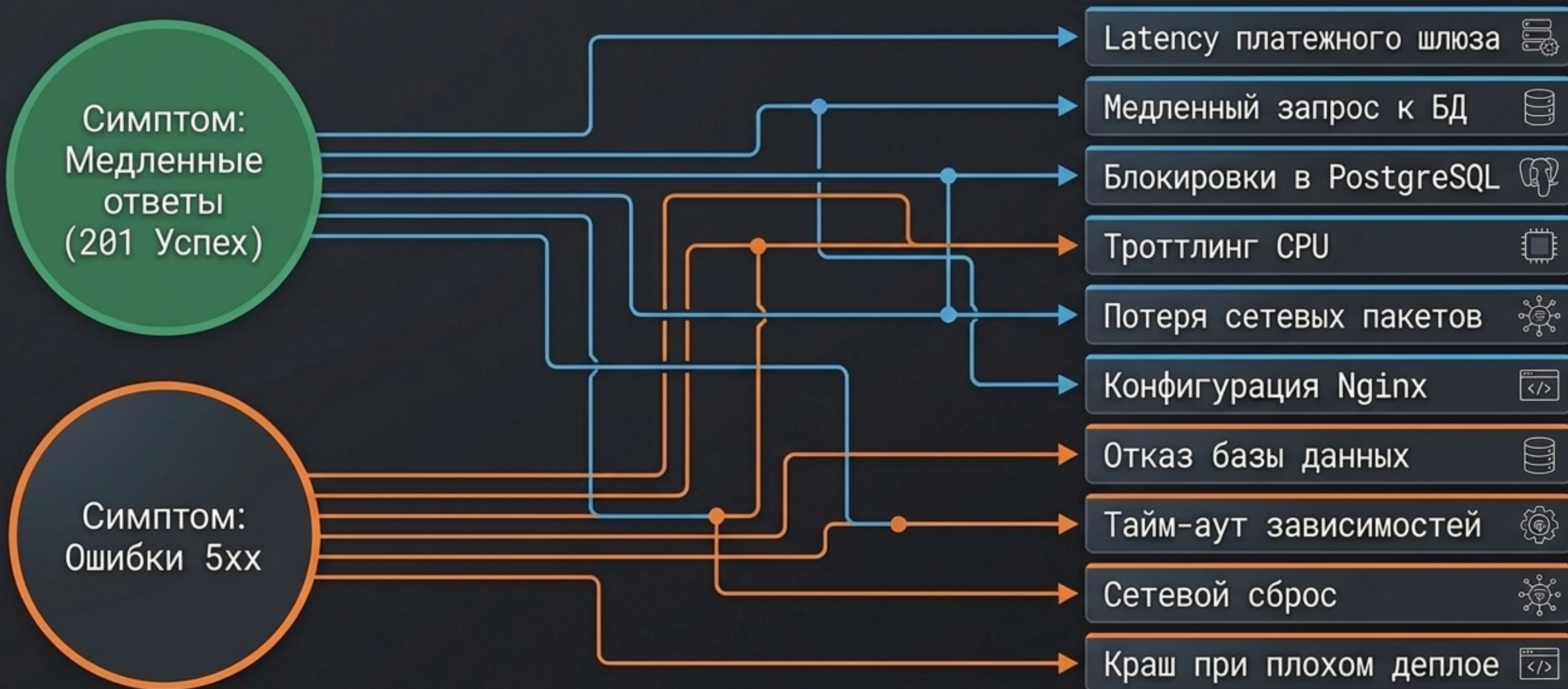


```
make recover
```

Аварийное снятие всех
активных фолтов

Иллюзия симптомов: Суть задачи SRE-агента

Один и тот же внешний симптом может быть вызван множеством разных причин.



Жизненный цикл эксперимента: Научный подход к хаосу



Анатомия вердикта: Запуск make experiment

{

Ground Truth (Скрыто)

Истинная причина сооя

CLASSIFIED

Incident (То, что расследуется)

Симптомы, время начала, пораженные домены

VISIBLE TO AGENT

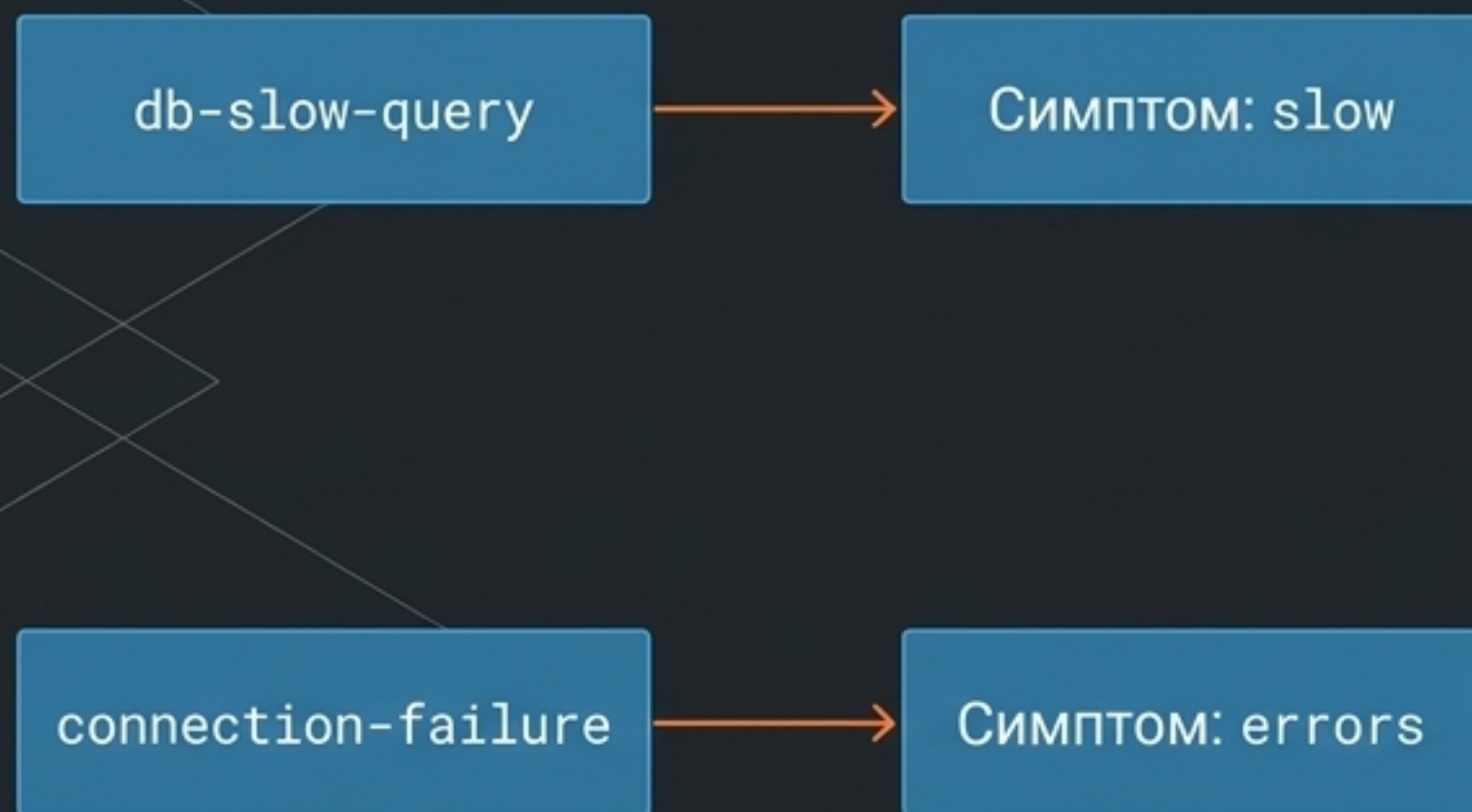
Verdict (Оценка)

- ✓ expected_seen: Ожидаемый симптом
- ✓ recovered: Восстановление системы
- ✓ evidence_matched: Совпадение доменов

}

Каталог сценариев: От базовых сбоев до M8-Челленджей

Простые сценарии (Изолированные)



Комплексные сценарии (M8-Challenges)



Одинаковый симптом: 8 разных сценариев выдают $p95 > 2c$ без ошибок. Отличаются только микро-уликами.



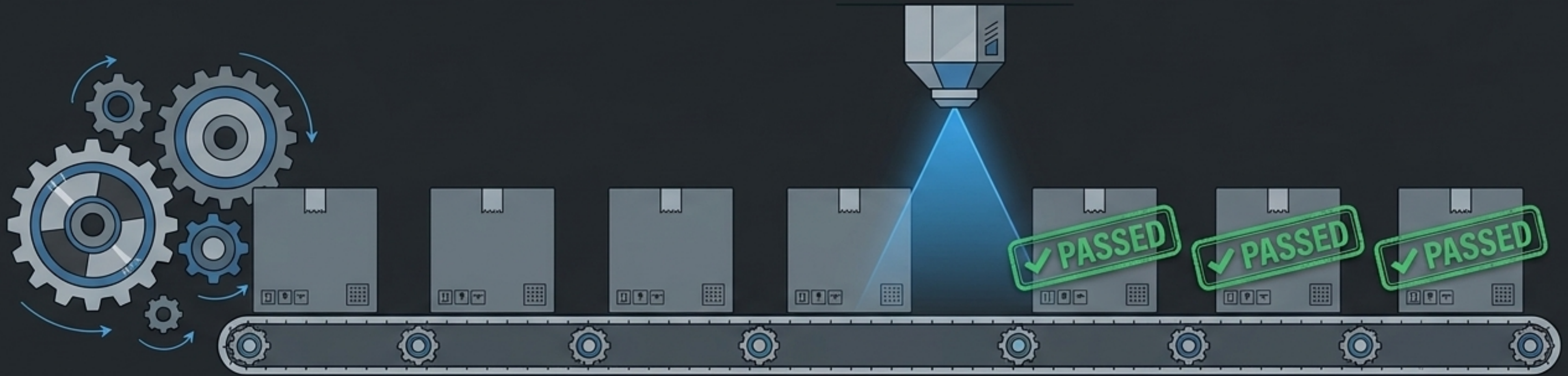
Ложная корреляция: Безвредный деплой, а через 5 минут падает смежный сервис. Откат не решает проблему.



Разрушение модели: Все метрики в норме, но клиенты страдают. Проблема спрятана в слепом пятне Nginx.

Гарантия качества: Автоматизированная приёмка

Встроенные механизмы для валидации стенда и агентов



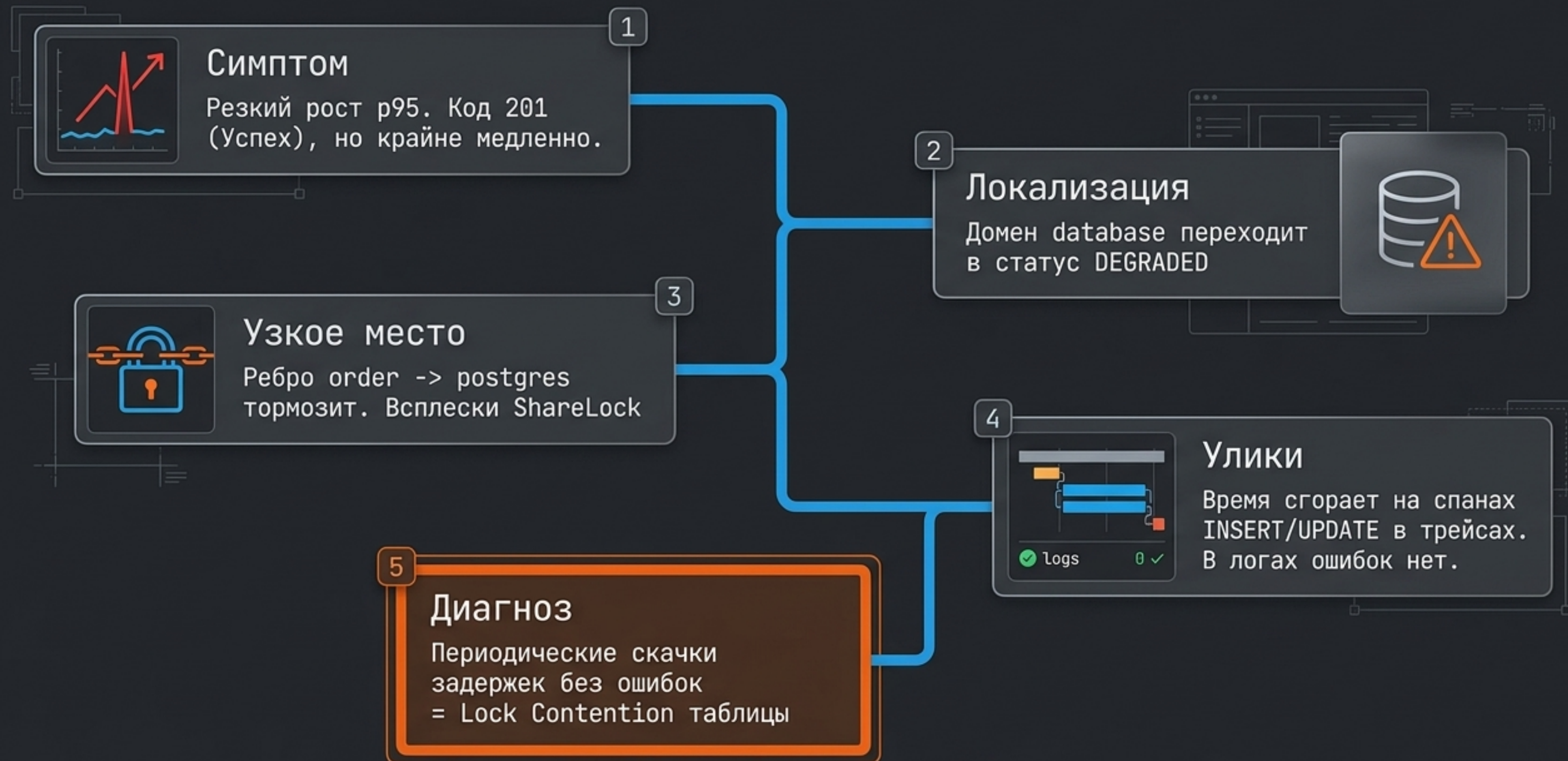
make acceptance

Прогон всех сценариев подряд (~30 мин).
Опция `FULL=1` для полных длительностей.
Опция `ROUNDS=2` для проверки детерминированности.

make test

Изоляция фолтов -> Интеграция -> Полные эксперименты -> Челленджи M8.

Мастер-класс: Расследование инцидента



Кастомизация среды: Файл конфигурации (.env)

Единый центр управления, читаемый Docker Compose и Makefile.



```
NGINX_PORT=8080
GRAFANA_PORT=3000
PROMETHEUS_PORT=9090
FAULT_INJECTOR_PORT=8090
TOKEN_TTL_SECONDS=3600
GATEWAY_HTTP_TIMEOUT_SECONDS=8
DB_POOL_MAX_SIZE=30
DEMO_USERS=alice:alice,bob:bob
```



Жизненный цикл полигона: Очистка и Траблшутинг

Управление состоянием

`make recover`

Снимает фолты, данные сохраняются.

`make down`

Остановка контейнеров. Фолты останутся в томах.

`make reset`



Полное уничтожение (БД, телеметрия) и чистый старт с нуля.

Решение проблем

 Конфликт (409)	Идет другой эксперимент. Вызовите <code>make recover</code> .
 Ошибка применения (502)	Инъектор сам откатит изменения.
 Мас уснул на долгом тесте	Запускайте через <code>caffeinate -i make acceptance</code>

Шпаргалка оператора: Единый пульт

Запуск и Статус

```
make up
make smoke
make ps
make logs
```

Контрольно

```
make up
```

Генерация нагрузки

```
make load RATE=5 DURATION=120
```

Инъекции и Сброс

```
make fault TYPE=<тип> TARGET=<цель> PARAMS='{...}'
make faults
make recover
```

Эксперименты и Приёмка

```
make experiment SCENARIO=<имя>
make experiments
make acceptance FULL=1
make test
```

Радары (127.0.0.1)

```
8080 (API)
3000 (Grafana)
9090 (Prom)
8090 (Injector)
8091 (Runner)
```